Consignes

Travail à réaliser

Sur base de cette première version du cahier des charges, vous devez réaliser les tâches suivantes :

1. Créer les scripts SQL de votre backend qui implémentent les règles métier à l'aide de contraintes et de *triggers*.
2. Tester vos règles métier manuellement ou grâce à des scripts SQL de tests unitaires (des exemples vous seront fournis dans le projet de base).

Le cahier des charges sera complété dans une seconde partie avec la partie *endpoints* du backend (y compris les règles d'accès) et la partie front-end (Flutter).

Structuration des fichiers backend

Voici une proposition de structuration des fichiers de votre backend :

```
01_init.sql
02_security.sql
03_tables_and_data.sql  -- Création des tables (hors users) et insertion des
données de test
04.00_checks.sql        -- Contraintes de check pour les règles métier
invariantes (hors BR-xx)
04.<xx>_br_<aaaa>.sql   -- Triggers pour les règles métier invariantes
numérotées (BR-xx)
                        -- où <aaaa> est un nom court décrivant la règle
```

FAQ

Tout au long du projet, nous publierons des réponses à des questions ou des précisions pour le projet. N'oubliez pas d'en vérifier régulièrement le contenu (voir Moodle).

Modalités pratiques et consignes

Le projet sera réalisé par **groupe de deux ou trois étudiants** conformément aux **technologies vues au cours** (PostgreSQL, PostgREST, Dart, Flutter). Le développement sera réalisé de manière **collaborative** par **tous les étudiants** du groupe.

Votre projet doit cibler principalement le *device* **Windows (Desktop)** et doit pouvoir s'exécuter sur un PC de l'école.

L'utilisation d'un **repository GIT BitBucket** est obligatoire (il sera créé et rendu accessible par le chargé de cours). Son utilisation **correcte** et **régulière** (au minimum un commit par session de travail, libellés clairs pour les commits, permettant de suivre l'historique du projet, ...) par tous les membres de l'équipe fera partie des critères d'évaluation. L'utilisation de branches est laissée libre et à l'appréciation de chaque étudiant (sauf la branche `master` qui est obligatoire). Pour les messages des commits, vous devez respecter la convention de nommage décrite ci-dessous.

Si vous le souhaitez, vous pouvez ajouter des **fonctionnalités supplémentaires après accord du chargé de cours**. Celles-ci seront valorisées.

La version finale de chaque itération de votre projet fera l'objet d'un commit dans la branche `master` dans BitBucket et sera marquée par un **Tag**. Elle sera constituée des éléments obligatoires suivants :

- le **code source** de l'application, sous forme d'un projet respectant la structure de fichiers vue au cours et compatible avec l'environnement de développement IntelliJ ;
- tous les fichiers externes utilisés par votre application ;
- dans le fichier `README.md` de votre projet, une **note de livraison** destinée à fournir au chargé de cours des renseignements que vous jugerez utile de porter à son attention. Au minimum vous y indiquerez les noms des utilisateurs et leur mot de passe, les bugs connus, les fonctionnalités pas implémentées ou implémentées partiellement et les fonctionnalités supplémentaires.
- la **grille d'auto-évaluation** ; cette grille et les consignes correspondantes vous seront fournies dans le courant du projet.

Afin de rendre l'installation de votre application sur une autre machine la plus simple possible, votre dépôt BitBucket sera initialisé avec le code source de base qui respecte les règles suivantes (vous veillerez à ne pas les enfreindre pendant vos développements) :

- L'URL de base de votre backend sera `http://localhost:3000/`.
- Le nom de votre projet sera `prbd-2526-xyy`, où `x` représente la lettre de votre classe (`a`, `c`, `f` ou `g`) et `yy` représente le numéro de votre projet exprimé sur deux chiffres (ex: classe `A`, projet `2` => `http://localhost/prbd-2526-a02/`).
- La base de données doit être créée dans un sous-dossier `data` de votre projet et portera le nom `prbd-2526-xyy` et votre backend sera créé dans le schéma `public`.

Convention de nommage des commits

Vous devez respecter la convention de nommage des commits décrite ci-dessous. Elle est inspirée de la convention de nommage utilisée pour le projet Angular.

Les messages de vos commits doivent respecter la structure suivante :

```
<type>(<scope>): <short summary>
  |       |             |
  |       |             └⫸ Résumé de ce que vous avez mis dans le commit.
  |       |
  |       └⫸ Commit Scope: le nom de la fonctionnalité concernée par ce
commit | rien.
  |
  └⫸ Commit Type: build|docs|feat|fix|perf|refactor
```

`<type>` doit prendre une des valeurs suivantes :

- `build` : changements qui affectent les options du projet ou les dépendances externes (librairies)
- `docs` : changements qui concernent la documentation (analyse, readme, ...)
- `feat` : développement d'une nouvelle fonctionnalité
- `fix` : correction d'un bug
- `perf` : modification qui améliorer la performance
- `refactor` : modification qui se contente de restructurer le code sans corriger un bug ni apporter une nouvelle fonctionnalité

`<scope>` doit être utilisé dans le cas où les changements concernent une fonctionnalité principale (avec d'éventuels impacts sur d'autres fonctionnalités). Si le changement n'est pas lié à une fonctionnalité, mais est plus général, le scope (et les parenthèses) peut être omis.

Groupes de projet

Les projets sont dimensionnés pour être réalisables seul (au lieu de deux) ou à deux (au lieu de trois). Nous demandons malgré tout de les réaliser en équipes de deux ou trois pour favoriser l'apprentissage du travail de groupe et des outils collaboratifs.

Une fois les groupes de projet constitués, aucun changement n’est possible. Si un étudiant d’un groupe abandonne ou ne participe pas activement au projet, son (ou ses) binôme(s) le signalera(ont) au chargé de cours le plus rapidement possible.

Nous ne considérons pas qu'il y a moins de travail à faire quand on est à deux plutôt que seul (ou à trois plutôt qu'à deux). On peut en effet se répartir le travail, mais il y a une surcharge importante liée à la coordination (cfr votre expérience en TGPR), la compréhension et les tests de ce qu'a fait l'autre (les autres), des discussions, des avis divergents, de l’harmonisation du code, etc.

Le fait de se retrouver seul au lieu de deux (ou à deux au lieu de trois), en cours de projet, n'est donc pas un argument recevable pour justifier de n'être pas arrivé au bout du projet.

> **Important**
>
> Dans le cadre de notre projet, **tous les membres du groupe s'engagent à travailler en solidarité et à assumer collectivement la responsabilité de livrer les fonctionnalités demandées**. Chaque membre a un rôle essentiel à jouer, et la réussite du projet repose sur la contribution active de chacun.
>
> Il est important de préciser que, même si un membre venait à abandonner ou à fournir un travail inférieur aux attentes, le groupe est tenu de respecter le périmètre initial du projet et de livrer les résultats attendus. Dans ce cas, la personne concernée se verra attribuer une note inférieure, voire une note d'échec, en raison de son manque d'engagement.

Évaluation

Chaque itération sera évaluée et interviendra dans la note finale du projet. Nous vous demandons donc un **travail régulier**.

Le travail doit être réparti de manière **équitable sur l'ensemble des membres de l'équipe de projet**. Ceci sera vérifié, notamment, sur base de :

- la présence et la participation active pendant les cours ;
- la répartition des commits sur les différents utilisateurs (nombre de commits et leur répartition dans le temps) ; il est donc fondamental que vos commits vous soient attribués et pour cela n'oubliez pas de définir votre nom et votre email dans Git. Si vous travaillez en appliquant le *pair programming*[1], arrangez-vous pour que la répartition des commits entre vous soit la plus homogène possible.

L'évaluation finale du projet aura lieu fin mai / début juin. Une seconde session aura lieu fin juin.

Outre l'évaluation du projet chaque membre du groupe sera **interrogé individuellement** lors de l'examen pour valider sa maîtrise des technologies et de l'architecture utilisées dans le cadre du projet. Cet examen permettra de confirmer ou d'invalider la note de projet pour l'étudiant.

Pour réussir le cours, vous devez réussir le projet et l'examen.

L'examen, s'il est réussi, permet de moduler la note de projet obtenue en fonction du niveau de maîtrise démontré. S'il est raté, il entraîne un ajournement en première session et un refus en seconde session.

Si le projet présenté en première session est jugé trop peu avancé ou trop insuffisant, il n'y aura pas de défense individuelle et les étudiants du groupe seront ajournés en première session.

Si le chargé de cours estime qu'il est possible pour les étudiants de finaliser leur projet dans l'intervalle de temps entre les deux sessions, il le leur signalera. Dans le cas contraire, les étudiants n'auront pas accès à la seconde session.

La note individuelle de chaque étudiant sera fonction de la qualité du projet remis, de sa participation active et régulière et de la qualité de l'examen.

Si la participation d'un étudiant au projet pendant l'année est jugée insuffisante ou trop peu régulière, l'étudiant sera ajourné en première session et se verra refuser l'accès à la seconde session.

Plagiat

Ce travail doit être réalisé en groupes de projet. Chaque groupe produira une réalisation originale qui lui est propre. Toute tentative de réutilisation, même partielle, du travail d'un autre groupe (de la même classe ou d'une autre classe) ou d'une autre personne sera considérée comme du plagiat.

En cas de plagiat avéré, les projets concernés recevront une note nulle. Veuillez noter que le plagiat et la récidive de plagiat peuvent également faire l'objet de sanctions disciplinaires (voir le ROI du Bachelier en Informatique).

Charte IA

Pour rappel, ce projet est soumis au respect de la Charte IA que vous avez signée pour acceptation (voir Moodle).

Planning

Le projet complet devra être livrée vers la fin du mois de mai. Une date précise d'échéance vous sera communiquée plus tard.

Bon travail !

---

1. Le *Pair programming* est une méthode de travail dans laquelle deux développeurs travaillent ensemble sur un même poste de travail. La personne qui rédige le code est appelée conducteur (*driver*). La seconde personne, appelée observateur (*observer*), assiste le conducteur en décelant les imperfections, en vérifiant que le code implémente correctement le design et en suggérant des

alternatives de développement. Les rôles s'échangent régulièrement pendant la séance de programmation (Wikipedia). ↩